

AWS Scaling, Routing & Load Balancing

Comprehensive Master Notes for Production & Technical Interviews

1. The Timeouts & Delays (Clearing the Confusion)

It is very common to confuse the different "waiting periods" in AWS. Here is the ultimate cheat sheet on what waits for what, and why.

Feature	Where is it?	What does it do?	Industry Use Case
Deregistration Delay (Connection Draining)	Target Group (ALB/NLB)	Stops sending <i>new</i> requests to an instance, but waits for <i>active</i> requests to finish before completely deregistering it.	Graceful Shutdown: Waiting for a 5-minute video rendering job or an active checkout process to finish before the ASG safely terminates the EC2 instance.
Cooldown Period	ASG Configuration	The global "pause" button. After a scaling activity completes, it waits X seconds before evaluating metric alarms again.	Preventing "Thrashing": Stopping the ASG from compounding mistakes and launching 5 extra instances while the very first new one is still booting up and hasn't had time to lower the metric average yet.
Default Warm-up	ASG Metric Setting	Tells the ASG: "Don't include this specific instance's metrics toward the group's real-time average until it has been running for X seconds."	Accurate Metrics: An instance might spike to 100% CPU during its initial boot sequence while downloading scripts. Warm-up ignores this temporary spike so it doesn't trigger false scaling alarms.

2. The Instance Lifecycle & Hooks Masterclass

Crucial Architectural Rule: Health checks only begin once the instance reaches the InService state. Without lifecycle hooks, an instance transitions straight to InService, receives live traffic immediately, and instantly fails health checks because the underlying codebase hasn't finished booting or downloading.

1. Pending (Standard Boot)

EC2 instance is spun up by the EC2 API. Virtual hardware and base OS load.

2. Pending:Wait (Lifecycle Hook Pause)

The ASG holds the instance in a waiting pattern. The instance is **not** registered to the Target Group yet and receives zero public traffic.

3. UserData Script Executes

Your custom shell script runs in the background (e.g., pulling application builds from S3, retrieving production secrets). Once tasks finish successfully, the script calls the AWS CLI to complete the action.

4. Pending:Proceed

The ASG acknowledges the CONTINUE signal from your script and advances the instance's state.

5. InService (Traffic Ingestion)

The instance officially registers to the Target Group. **Load Balancer Health Checks begin.** As soon as it passes its first series of healthy checks, public traffic flows in.

Production-Grade UserData Script

This dynamic script allows the instance to seamlessly query its local environment via Instance Metadata Service Version 2 (IMDSv2) to safely signal completion to its parent ASG.

```
#!/bin/bash
# 1. Acquire the IMDSv2 Token (Mandatory Security Practice)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# 2. Extract local Instance ID and Region dynamically
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
REGION=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/placement/region)

# 3. Dynamically discover the parent ASG name using the internal AWS system tag
ASG_NAME=$(aws ec2 describe-tags --filters "Name=resource-id,Values=$INSTANCE_ID" "Name=key,Values=aws:autoscaling:groupName" --query "Tags[0].Value" --output text --region "$REGION")

# 4. Perform localized environment assembly (e.g., pull code, start app daemon)
systemctl start nginx

# 5. Signal the parent ASG to move the instance to the next phase
aws autoscaling complete-lifecycle-action --lifecycle-hook-name my-hook --auto-scaling-group-name "$ASG_NAME" --lifecycle-action-result CONTINUE --instance-id "$INSTANCE_ID" --region "$REGION"
```

3. Instance Refresh & Warm Pools

Instance Refresh (Rolling Updates)

What it is: A mechanism used to perform rolling replacements of an entire cluster (e.g., when a new launch template or golden AMI is published).

Production Strategy: Engineers configure a `MinHealthyPercentage` of 100%. This forces the ASG to bring up a fresh, updated node and verify its health **before** tearing down an old one, ensuring zero-downtime cluster upgrades.

Warm Pools

What it is: A repository of pre-initialized, configured instances resting in a `Stopped` or `Stopped/Running` state.

Why companies use it: If a production Java/Go application requires 10+ minutes to compile assets or warm up local caches at launch, reactive scaling is completely useless during a rapid flash sale. Warm Pools pre-bake the instances and stop them, keeping compute costs at absolute zero while dropping launch times from minutes down to a 30-second boot.

4. Advanced Traffic Routing (The "Mind-Blowing" Scenarios)

Scenario A: The Canary Release

Instead of deploying code across an entire cluster at once, you configure a single ALB listener rule to split traffic across two distinct target groups: **95% to TargetGroup-V1** and **5% to TargetGroup-V2 (The Canary Group)**. If error rates remain flat on V2, you seamlessly dial up the slider to 50/50 and then 0/100 without a single dropped packet. If an exception spikes, you drop V2's weight to 0% instantly.

Scenario B: Header / Device-Based Routing

Rather than making applications parse browser types, the ALB can inspect HTTP request components natively at layer 7. It can look at the User-Agent header: matching "iPhone" or "Android" routes requests directly to a microservice pool optimized for lightweight mobile payloads, while desktop traffic falls through to standard instances.

Scenario C: Serverless Fixed Responses & Redirects

If you need to handle HTTP-to-HTTPS redirection or render a beautiful "Under Maintenance" page, running dedicated Nginx servers just to handle redirects wastes money and adds operational overhead. An ALB can evaluate rules and generate a **Fixed Response (e.g., 503 HTTP status with custom JSON/HTML)** or a **Redirect (301)** directly at the load balancer level, requiring zero underlying compute power.

5. Scaling in the Real World: Actual Industry Policies

1. Target Tracking Policies

This is the standard engineering practice for everyday workloads. Rather than setting strict step alarms, you tell AWS: "Maintain the cluster's aggregate metric (like average CPU utilization or ALB Request Count Per Target) at precisely 50%." The ASG dynamically computes the necessary capacity additions or removals automatically as demand moves.

2. Predictive Scaling

For cyclical traffic structures (e.g., an enterprise tool spiking at 9:00 AM every Monday morning), standard reactive scaling is too slow because it reacts after the spike happens. Predictive scaling applies Machine Learning models over 14 days of historical application patterns to map expected loads and pre-provision scaling groups 15 to 30 minutes *before* the spike hits.

3. Mixed Instance Allocation & Spot Fleets

To optimize infrastructure spend drastically, production environments combine purchasing options inside a single ASG. You configure a base capacity of On-Demand instances to guarantee baseline performance, and specify that any scaling capacity above that baseline must be fulfilled using a diversified pool of Spot Instances. This saves up to 70-90% on infrastructure costs during traffic spikes.

6. Deep Dive: How ASG Actually Selects Subnets & AZs

A common misconception is that the ASG itself deploys structural components inside your network. In reality, an ASG is a **Logical Regional Controller** that stays in the background and issues API instructions to the EC2 engine.

The AZ Balancing Act (The Primary Law of ASG)

- The absolute top priority of an ASG is to maintain an exactly equal distribution of EC2 nodes across all chosen Availability Zones.
- If you select 3 subnets across 3 AZs (e.g., 1a, 1b, 1c) and set your desired capacity to 3, the ASG will strictly spin up 1 instance per zone. If capacity scales to 4, it will pick one zone to host the 4th instance (e.g., 2, 1, 1).

The Rebalancing Workflow

If a zone goes down or an instance is manually terminated, the group becomes unbalanced. To correct this, the ASG executes a **Launch-Before-Terminate** routine. It spins up a replacement node in the underrepresented AZ first, confirms its operational health, and then safely tears down any excess node in an overrepresented zone to avoid reducing live capacity during the transition.

The "Zone-Blocked" Placement Failure

If you attach a subnet to your ASG where a specific instance type is completely unsupported by AWS hardware (for example, a `t3.micro` requested inside `us-east-1e`), the ASG's balancing logic gets completely broken. Because it is fiercely trying to satisfy its cross-AZ balancing rules, it will repeatedly attempt to launch in the broken subnet, fail at the API layer, and get stuck in a loop. This leaves your warm pool or scaling group completely short of instances until that specific subnet is deselected from the ASG's configuration boundaries.

7. Network Layer Placements: ALB vs. NLB Subnet Mechanics

While an ASG is purely a logical controller, AWS Load Balancers operate entirely differently. When you select subnets for an ALB or an NLB, you are instructing AWS to provision physical structural points directly into your network.

Architecture Vector	Application Load Balancer (ALB)	Network Load Balancer (NLB)
Subnet Attachment Requirement	Strictly Requires a Minimum of 2 AZs. You must select at least two subnets, each residing in a completely separate Availability Zone.	Supports Single AZ Deployment. Can be pinned to operate inside a single subnet/AZ if your architecture requires it, though multi-AZ is recommended for production.
Network Footprint (ENIs)	AWS provisions internal **Elastic Network Interfaces (ENIs)** directly inside every selected subnet. These interfaces receive dynamic IP addresses from your subnet pool. Their IPs change as the ALB automatically scales horizontally.	AWS provisions highly optimized network nodes directly inside your subnets. Crucially, **NLB nodes support Static Elastic IPs.** Their IP addresses remain fixed forever, allowing upstream firewalls to safelist them explicitly.
Cross-Zone Load Balancing	Always Enabled by Default. If an ALB instance in 1a receives a request, it can natively route it to a target instance sitting in 1b to keep workload distribution smooth across all targets. No extra regional data transfer charges apply.	Disabled by Default. Traffic hitting the NLB node in 1a stays strictly within the targets of 1a. Enabling cross-zone routing introduces slight network latency variations and can incur cross-AZ data processing costs.
Use Case Selection Rule	Best for HTTP, HTTPS, Web Applications, and REST APIs where advanced path-based, header-based, or weighted routing is needed at Layer 7.	Best for raw TCP, UDP, TLS, gaming protocols, and massive high-performance pipelines handling millions of requests per second at Layer 4 with ultra-low microsecond latencies.

8. Interview "Gotcha" Scenarios

The "Poison Pill" Infinite Loop

The Setup: You configure your ASG to rely on **ELB Health Checks** instead of default EC2 status checks (meaning if the load balancer marks a node as unhealthy, the ASG will instantly mark it for termination).

The Problem: A developer accidentally pushes a broken code release where the internal health route (e.g., /health) returns a 500 error due to a minor syntax bug, but the server's OS is running perfectly fine.

The Catastrophic Result:

1. The ALB notes the 500 error and marks the instance as Unhealthy.
2. The ASG acts immediately, killing the instance and spinning up a replacement node.
3. The replacement node provisions, pulls the identical buggy codebase, and boots up.
4. The ALB marks the new node Unhealthy as well.
5. The ASG kills it again—creating an ****infinite, rapid loop of instance terminations and launches**** that can destroy application availability and run up enormous unexpected AWS bills.

The Production Fix: In an active emergency, navigate directly to your ASG -> Advanced Configurations -> Suspended Processes, and select ****Suspend ReplaceUnhealthy****. This instantly freezes the loop, preserving the broken nodes in place so your engineering team can safely SSH in, extract logs, and patch the root cause.